

Experiment No.5

Title: Implement Stack and Queue using arrays.

Aim: :Implement stack, using array and linked list and perform operations like PUSH, POP, and DISPLAY contents of the stack. Also display position of TOP

Name of Student: Amey Pandit

Class:ET

Div:3

PRN No:B25ET1175

Batch:B

// Stack using array

```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 5
struct stack
{
    int s[SIZE];
    int top;
} st;
int isFull()
{
    if (st.top >= SIZE - 1)
        return 1;
    else
        return 0;
}
int isEmpty()
{
    if (st.top == -1)
        return 1;
    else
        return 0;
}
int pop()
{
    int item;
    item = st.s[st.top];
    st.top--;
    return item;
}
```

```

void push(int item)
{
    st.top++;
    st.s[st.top] = item;
}

void display()
{
    int i;
    if (isEmpty())
        printf("\nStack is empty!!!");
    else
    {
        for (i = st.top; i >= 0; i--)
            printf("\n%d", st.s[i]);
    }
}

int main(void)
{
    int item, choice;
    char ans;
    st.top = -1;
    printf("\nImplementation of Stack");
    do
    {
        printf("\nMain menu");
        printf("\n1. Push\n2. Pop\n3. Display\n4. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                printf("\nEnter the item to be pushed: ");
                scanf("%d", &item);
                if (isFull())
                    printf("\nStack is full");
                else
                    push(item);
                break;
            case 2:
                if (isEmpty())

```

```

printf("\nEmpty stack (Underflow)");
else
{
item = pop();
printf("\nThe popped element is: %d", item);
}
break;
case 3:
display();
break;
case 4:
exit(0);
}
printf("\nDo you want to continue? (Y/N): ");
scanf(" %c", &ans);
} while (ans == 'Y' || ans == 'y');
return 0;
}

```

//QUEUE using array

```

#include <stdio.h>
#include <stdlib.h>
#define SIZE 5
struct Queue {
int arr[SIZE];
int front, rear;
} q;
int isfull() {
if (q.rear >= SIZE - 1)
return 1;
else
return 0;
}
void insert(int item) {
if (isfull()) {
printf("\n Queue is full");
} else {
q.rear = q.rear + 1;

```

```

q.arr[q.rear] = item;
}
}
int isempty() {
if (q.front > q.rear)
return 1;
else
return 0;
}
int delete() {
int item;
if (isempty()) {
printf("\n Queue is Empty");
return -1;
} else {
item = q.arr[q.front];

q.front = q.front + 1;
return item;
}
}
void display() {
int i;
if (isempty()) {
printf("\n Queue is Empty");
} else {
for (i = q.front; i <= q.rear; i++) {
printf("%d\t", q.arr[i]);
}
}
}
void main() {
int choice, item;
q.front = 0;
q.rear = -1;
while (1) {
printf("\n Implementation of queue");
printf("\n 1. Insert");
printf("\n 2. Delete");
printf("\n 3. Display");

```

```
printf("\n 4. Exit");
printf("\n Enter Choice: ");
scanf("%d", &choice);
switch (choice) {
case 1:
printf("\n Enter item to insert: ");
scanf("%d", &item);
insert(item);
break;
case 2:
if (isempty()) {
printf("\n Queue is Empty");
} else {
item = delete();
}
break;
case 3:
display();
break;
case 4:
printf("\n Exiting the program...");
exit(0);
default:
printf("\n Invalid Choice !!");
}
}
}
```